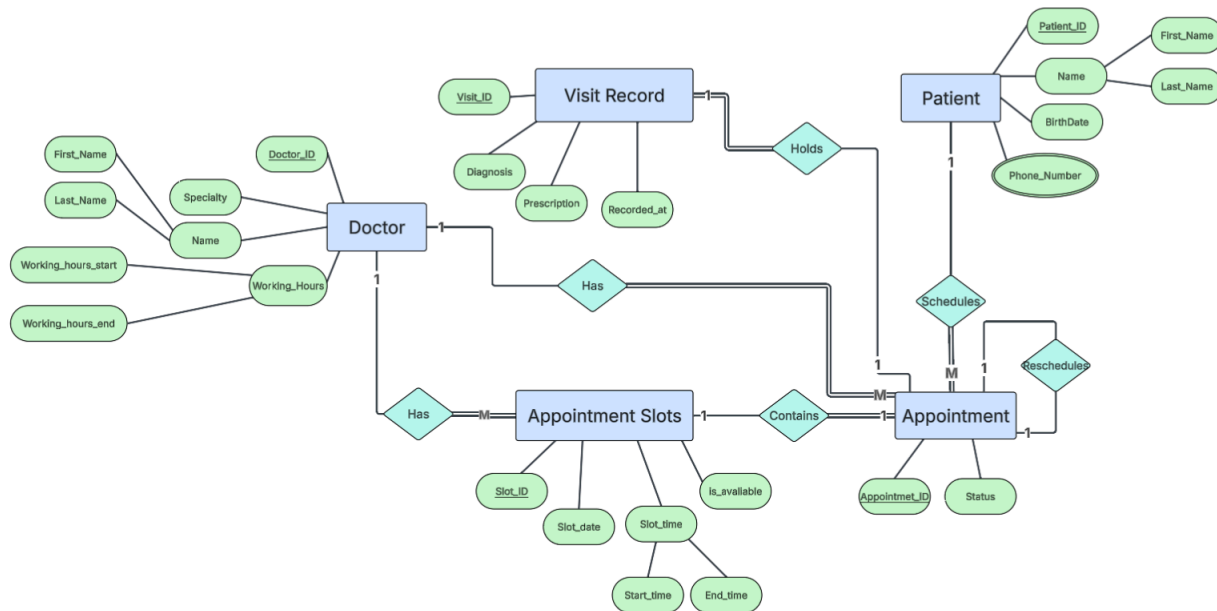
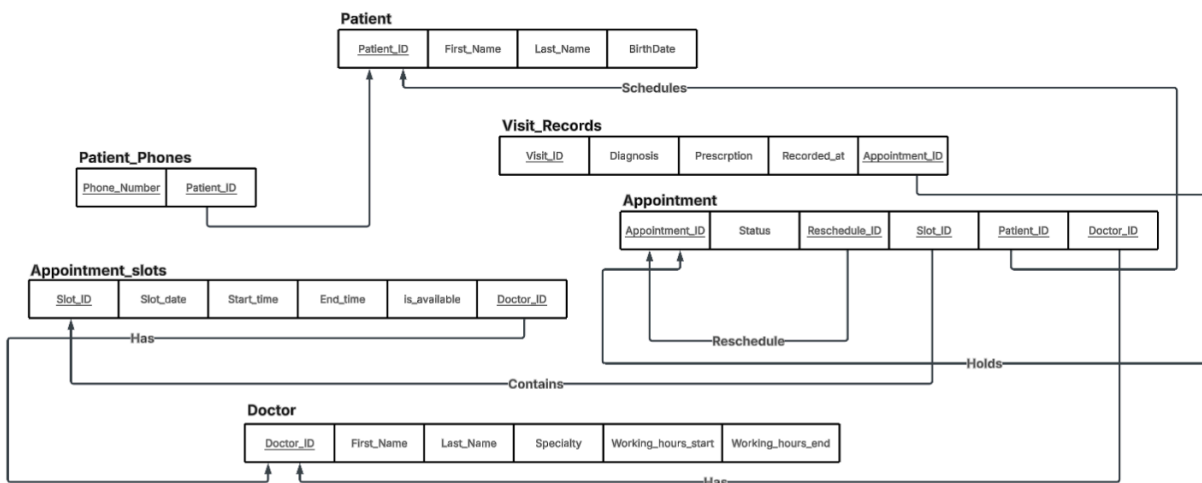


Clinic Appointment & Patient Records API

ERD & Mapping



Activate Windows
Go to Settings to activate Windows.



To solve the cancellation-without-deletion problem I created an entity called appointment where all appointments are recorded. A newly created appointment is labelled as a scheduled appointment if that appointment is rescheduled then it's assigned a reschedule id of the old appointment and a new appointment id, cancelled appointments are just labelled cancelled and a new appointment should be made if the user wishes to do so. Appointment_slots entity contains all the active (scheduled) appointments. So Appointment entity acts as a history for all appointment statuses while appointment slot is only a history for scheduled appointments.

Testing

Creating a patient:

The screenshot shows a REST client interface with the following components:

- Header:** [CONFLICT] POST Get da + No environment
- Toolbar:** My Collection > Get data Save Share
- Request Section:**
 - Method:** POST
 - URL:** http://localhost:8080/patients
 - Body:** raw JSON
 - Request Body:**

```
1 {  
2   "firstName": "Khadija",  
3   "lastName": "Al-Mamari",  
4   "birthDate": "2000-03-15",  
5   "phones": ["98772211", "79112034"]  
6 }
```
- Response Section:**
 - Status:** 201 Created
 - Time:** 194 ms
 - Size:** 281 B
 - Body:** JSON
 - Response Body:**

```
1 {  
2   "id": 21,  
3   "firstName": "Khadija",  
4   "lastName": "Al-Mamari",  
5   "birthDate": "2000-03-15",  
6   "phones": [  
7     "98772211",  
8     "79112034"  
9   ]  
10 }
```

Getting a specific patient:

The screenshot shows an HTTP client interface with the following details:

- Collection:** My Collection > Get data
- Method:** GET
- URL:** http://localhost:8080/patients/1
- Buttons:** Save, Share, Send
- Body:** raw (selected), JSON, Schema, Beautify
- Response:** 200 OK, 52 ms, 280 B
- JSON Response:**

```
1 {
2   "id": 1,
3   "firstName": "Ahmed",
4   "lastName": "Al-Balushi",
5   "birthDate": "1990-03-15",
6   "phones": [
7     "96812345678",
8     "96887654321"
9   ]
10 }
```

Creating a doctor:

The screenshot displays a REST client interface with the following components:

- Header:** Shows a status bar with "[CONFLICT] POST Get da", a dropdown menu, and "No environment".
- Toolbar:** Includes "My Collection > Get data", "Save", and "Share" buttons.
- Request Configuration:** A dropdown menu set to "POST" and a text field containing the URL "http://localhost:8080/doctors". A "Send" button is to the right.
- Body Editor:** Labeled "Body", it has tabs for "raw" and "JSON" (selected). It includes "Schema" and "Beautify" options. The JSON body is:

```
1 {
2   "firstName": "Khalid",
3   "lastName": "Al-Salmi",
4   "speciality": "Ortho",
5   "workingHoursStart": "10:00",
6   "workingHoursEnd": "19:00"
7 }
```
- Response Section:** Labeled "Body", it shows a green status bar "201 Created" with "22 ms" and "302 B". Below it, a "JSON" tab is active, displaying the response body:

```
1 {
2   "id": 24,
3   "firstName": "Khalid",
4   "lastName": "Al-Salmi",
5   "speciality": "Ortho",
6   "workingHoursStart": "10:00:00",
7   "workingHoursEnd": "19:00:00"
8 }
```

Getting a specific doctor:

The screenshot shows a REST client interface with the following components:

- Header:** HTTP icon, "My Collection > Get data", "Save" button, "Share" button.
- Request Section:** Method dropdown set to "GET", URL input field containing "http://localhost:8080/doctors/2", and a blue "Send" button.
- Body Section:** "Body" label with a dropdown, "Cookies" link, "raw" dropdown, "JSON" dropdown, "Schema" link, and "Beautify" link.
- Response Section:** "Body" label with a refresh icon, status bar showing "200 OK • 74 ms • 315 B • [globe icon] | e.g. ...", and a JSON viewer with a "{ } JSON" dropdown and various icons.

The response body is displayed in a code editor with line numbers 1 through 8, showing a JSON object for a doctor:

```
1 {
2   "id": 2,
3   "firstName": "Mohammed",
4   "lastName": "Al-Balushi",
5   "speciality": "Ear, Nose and Throat",
6   "workingHoursStart": "10:00:00",
7   "workingHoursEnd": "20:00:00"
8 }
```

Doctor that does not exist yet:

The screenshot shows a REST client interface with the following details:

- Tab:** [CONFLICT] GET Get dat
- Environment:** No environment
- Collection:** My Collection > Get data
- Method:** GET
- URL:** http://localhost:8080/doctors/3/sched ...
- Buttons:** Save, Share, Send
- Body:** raw (selected), JSON, Schema, Beautify
- Response:** 404 Not Found (11 ms, 245 B)
- Response Body (JSON):**

```
1 {
2   "status": 404,
3   "error": "Not Found",
4   "message": "Doctor not found with ID: 3"
5 }
```

Patient that does not exist yet:

The screenshot shows a REST client interface with the following components:

- Header:** "My Collection > Get data" with "Save" and "Share" buttons.
- Request Bar:** Method "GET" and URL "http://localhost:8080/patients/31" with a "Send" button.
- Body Tab:** Labeled "Body" with a dropdown set to "raw" and "JSON" selected. It includes "Schema" and "Beautify" links.
- Response Area:** A large empty box with a "1" in the top left corner.
- Status Bar:** Shows "404 Not Found" in red, followed by "8 ms", "247 B", a globe icon, and a link icon.
- Response Body:** A JSON object displayed in a code editor:

```
1 {
2   "status": 404,
3   "error": "Not Found",
4   "message": "Patient not found with ID: 31"
5 }
```

Doctor with free schedule:

 My Collection > **Get data**  Save  Share

GET 

http://localhost:8080/doctors/23/schei...

Send 

Body  

raw 

JSON 

 Schema 

1

Body  

200 OK • 10 ms • 166 B •  |  e.g. ... 

{ } JSON 

 |   |  

1

[]

Doctor schedule with patients:

The screenshot shows a web browser's developer tools interface. At the top, the 'My Collection > Get data' tab is active. The request method is 'GET' and the URL is 'http://localhost:8080/doctors/23/sche...'. The 'Send' button is visible. Below the request bar, the 'Body' tab is selected, showing a 'raw' JSON response. The response is a JSON array with two objects, each representing a doctor appointment slot. The first object has a 'slotId' of 42, a 'startTime' of '09:00:00', an 'endTime' of '09:30:00', is 'available: false', has an 'appointmentId' of 22, and a 'patientName' of 'Khadija Al-Mamari'. The second object has a 'slotId' of 43, a 'startTime' of '13:00:00', an 'endTime' of '13:30:00', is 'available: false', has an 'appointmentId' of 23, and a 'patientName' of 'Salwa Al-Busaidi'. The status bar at the bottom indicates a '200 OK' response with a response time of 18 ms and a size of 422 B.

```
1 [
2   {
3     "slotId": 42,
4     "startTime": "09:00:00",
5     "endTime": "09:30:00",
6     "available": false,
7     "appointmentId": 22,
8     "patientName": "Khadija Al-Mamari"
9   },
10  {
11    "slotId": 43,
12    "startTime": "13:00:00",
13    "endTime": "13:30:00",
14    "available": false,
15    "appointmentId": 23,
16    "patientName": "Salwa Al-Busaidi"
17  }
18 ]
```


Patient with no previous visit history:

The screenshot shows a web client interface with the following components:

- Header:** "My Collection > Get data" with "Save" and "Share" buttons.
- Request Bar:** Method "GET" and URL "http://localhost:8080/patients/2/history". A "Send" button is on the right.
- Body View:** A dropdown menu shows "raw" selected, with "JSON" as an alternative. "Schema" and "Beautify" links are also present.
- Response Area:** A large text area containing the number "1".
- Status Bar:** Displays "200 OK", "10 ms", "166 B", and a "e.g." link.
- Footer:** A "JSON" dropdown, a play button, and a list of icons (refresh, list, search, copy, link).

Booking an appointment:

[CONFLICT] POST Get da +

No environment

My Collection > Get data Save Share

POST http://localhost:8080/appointments Send

Body Cookies

raw JSON Schema Beautify

```
1 {
2   "patientId": 21,
3   "doctorId": 23,
4   "date": "2026-07-10",
5   "startTime": "09:00"
6 }
```

Body 201 Created • 58 ms • 407 B • e.g. ...

{ } JSON JSON

```
1 {
2   "appointmentId": 22,
3   "status": "SCHEDULED",
4   "patientId": 21,
5   "patientName": "Khadija Al-Mamari",
6   "doctorId": 23,
7   "doctorName": "Ahmed Al-Hamdani",
8   "slotDate": "2026-07-10",
9   "startTime": "09:00:00",
10  "endTime": "09:30:00",
11  "rescheduledToAppointmentId": null
12 }
```

Cancelling an appointment:

The screenshot shows a REST client interface with the following details:

- Collection:** My Collection > Get data
- Method:** POST
- URL:** http://localhost:8080/appointments/8/...
- Body:** raw (JSON)
- Response:** 200 OK, 85 ms, 401 B
- Response Body (JSON):**

```
1 {
2   "appointmentId": 8,
3   "status": "CANCELLED",
4   "patientId": 2,
5   "patientName": "Salwa Al-Busaidi",
6   "doctorId": 2,
7   "doctorName": "Mohammed Al-Balushi",
8   "slotDate": "2026-07-15",
9   "startTime": "10:35:00",
10  "endTime": "11:05:00",
11  "rescheduledToAppointmentId": null
12 }
```

Cancelling an appointment that isnt scheduled:

The screenshot shows a REST client interface with the following elements:

- Header:** [CONFLICT] POST Get da, No environment, Save, Share.
- Method and URL:** POST, http://localhost:8080/appointments/1/c...
- Body:** raw, JSON, Schema, Beautify. The body content is "1".
- Response:** 400 Bad Request, 13 ms, 264 B. The response body is a JSON object:

```
{  "status": 400,  "error": "Bad Request",  "message": "Only SCHEDULED appointments can be cancelled. Current status: CANCELLED"}
```

Rescheduling an appointment:

The screenshot shows a REST client interface with the following components:

- Top Bar:** Includes a "My Collection > Get data" breadcrumb, "Save" and "Share" buttons, and a "Send" button.
- Request Section:** Shows a "POST" method and the URL "http://localhost:8080/appointments/23...".
- Body Section:** The "Body" tab is selected, showing a JSON payload in "raw" format:

```
1 {
2   "newDate": "2026-07-15",
3   "newStartTime": "11:00"
4 }
```
- Response Section:** The "Body" tab is selected, showing a "200 OK" status with a response time of 44 ms and a size of 400 B. The JSON response is displayed in "JSON" format:

```
1 {
2   "appointmentId": 24,
3   "status": "SCHEDULED",
4   "patientId": 2,
5   "patientName": "Salwa Al-Busaidi",
6   "doctorId": 23,
7   "doctorName": "Ahmed Al-Hamdani",
8   "slotDate": "2026-07-15",
9   "startTime": "11:00:00",
10  "endTime": "11:30:00",
11  "rescheduledToAppointmentId": null
12 }
```

Rescheduling an appointment that isn't scheduled:

The screenshot shows a REST client interface with the following details:

- Header:** [CONFLICT] POST Get da, No environment.
- Toolbar:** My Collection > Get data, Save, Share.
- Request:** Method: POST, URL: http://localhost:8080/appointments/1/r..., Send button.
- Body:** raw, JSON, Schema, Beautify. The body content is a single line: 1.
- Response:** Body, 500 Internal Server Error, 5 ms, 241 B. The response is displayed in JSON format:

```
1 {
2   "status": 500,
3   "error": "Internal Server Error",
4   "message": "An unexpected error occurred"
5 }
```


Completing an appointment:

The screenshot shows a REST client interface with the following components:

- Top Bar:** Includes an HTTP icon, "My Collection > Get data", a "Save" button, and a "Share" button with a link icon.
- Request Section:** A dropdown menu shows "POST", and the URL field contains "http://localhost:8080/appointments/21 ...". A blue "Send" button is to the right.
- Body Section:** Labeled "Body" with a dropdown arrow. It contains a "raw" dropdown and a "JSON" dropdown. To the right are "Schema" and "Beautify" links.
- Response Section:** A large text area shows the number "1".
- Status Bar:** Displays "Body" with a refresh icon, a green status "200 OK", and response details: "27 ms", "402 B", a globe icon, "e.g.", and a three-dot menu.
- JSON View:** A dropdown shows "{} JSON" with a play icon and a screenshot icon. To the right are icons for expand/collapse, search, copy, and share.
- JSON Data:** A pre-formatted JSON object is displayed with line numbers 1 through 12:

```
1 {  
2   "appointmentId": 21,  
3   "status": "COMPLETED",  
4   "patientId": 2,  
5   "patientName": "Salwa Al-Busaidi",  
6   "doctorId": 2,  
7   "doctorName": "Mohammed Al-Balushi",  
8   "slotDate": "2026-07-10",  
9   "startTime": "10:00:00",  
10  "endTime": "10:30:00",  
11  "rescheduledToAppointmentId": null  
12 }
```

Completing an appointment that isn't scheduled:

The screenshot shows a REST client interface with the following components:

- Header:** "My Collection > Get data" with "Save" and "Share" buttons.
- Method and URL:** A dropdown menu set to "POST" and a text field containing "http://localhost:8080/appointments/1/c...". A "Send" button is to the right.
- Body Tab:** Labeled "Body" with a dropdown menu set to "raw" and "JSON". To the right are "Schema" and "Beautify" links.
- Request Body:** A large text area containing the number "1".
- Response Section:** Below the request body, it shows "Body" with a refresh icon, a status bar indicating "400 Bad Request", and performance metrics: "41 ms", "274 B", and a globe icon.
- Response Body:** A JSON object displayed in a code editor with line numbers 1 through 5. The JSON is:

```
{  "status": 400,  "error": "Bad Request",  "message": "Only SCHEDULED appointments can be marked as completed. Current status: CANCELLED"}
```

Recording a visit:

 My Collection > Get data  Save  Share

POST  http://localhost:8080/appointments/21 ...  Send 

Body  Cookies

raw  JSON  Schema Beautify

```
1  {
2    "diagnosis": "Hypertension Stage 1",
3    "prescription": "Panadol 20mg once daily for 30
4    days"
}
```

Body  201 Created • 64 ms • 405 B •  e.g. ...

{ } JSON         

```
1  {
2    "visitId": 21,
3    "appointmentId": 21,
4    "patientId": 2,
5    "patientName": "Salwa Al-Busaidi",
6    "visitDate": "2026-07-10",
7    "diagnosis": "Hypertension Stage 1",
8    "prescription": "Panadol 20mg once daily for 30
9    days",
10   "recordedAt": "2026-06-23T14:42:21.9510179"
}
```

Trying to do visit for not a completed appointment:

The screenshot shows a REST client interface with the following details:

- Header:** [CONFLICT] POST Get da, No environment.
- Toolbar:** HTTP icon, My Collection > Get data, Save, Share.
- Request:** Method: POST, URL: http://localhost:8080/appointments/23..., Send button.
- Body:** Tabbed view with 'raw' and 'JSON' options. The 'JSON' tab is active, showing a valid JSON object:

```
1 {
2   "diagnosis": "Hypertension Stage 1",
3   "prescription": "Panadol 20mg once daily for 30
4     days"
5 }
```
- Response:** Status: 400 Bad Request, 18 ms, 275 B. The response body is shown in a JSON tab:

```
1 {
2   "status": 400,
3   "error": "Bad Request",
4   "message": "Visit can only be recorded for
5     COMPLETED appointments. Current status:
6     RESCHEDULED"
7 }
```

Business Logic

Working hours of a doctor must be before working hours end:

The screenshot shows a REST client interface. At the top, the collection is 'My Collection' and the method is 'POST' to the URL 'http://localhost:8080/doctors'. The request body is a JSON object representing a doctor's information. The response is a '400 Bad Request' with a message indicating that the working hours start must be before the working hours end.

Request:

```
POST http://localhost:8080/doctors
```

Body (JSON):

```
{
  "firstName": "Sarah",
  "lastName": "Al-Rashdi",
  "speciality": "Cardiology",
  "workingHoursStart": "16:00",
  "workingHoursEnd": "08:00"
}
```

Response:

```
400 Bad Request - 7 ms - 245 B
```

```
{
  "status": 400,
  "error": "Bad Request",
  "message": "Working hours start must be before working hours end"
}
```

Appointment date must not be in the past of the current time and date:

The screenshot shows a REST client interface with the following details:

- Tab:** [CONFLICT] POST Get da
- Environment:** No environment
- Collection:** My Collection > Get data
- Method:** POST
- URL:** http://localhost:8080/appointments
- Body (raw JSON):**

```
1 {
2   "patientId": 22,
3   "doctorId": 21,
4   "date": "2026-06-10",
5   "startTime": "09:00"
6 }
```
- Status:** 400 Bad Request (22 ms, 245 B)
- Response Body (JSON):**

```
1 {
2   "status": 400,
3   "error": "Validation Failed",
4   "message": "date: Appointment date must not be
5             in the past"
```

An appointment shouldnt be booked outside of doctors working hours:

The screenshot shows a REST client interface with a tab labeled "[CONFLICT] POST Get da". The request is a POST to "http://localhost:8080/appointments". The request body is a JSON object:

```
1 {  
2   "patientId": 21,  
3   "doctorId": 21,  
4   "date": "2026-07-10",  
5   "startTime": "23:00"  
6 }
```

The response is a "400 Bad Request" with a status of 400, a message "Requested time 23:00 is outside the doctor's working hours (10:00 - 18:00)", and an error "Bad Request".

```
1 {  
2   "status": 400,  
3   "error": "Bad Request",  
4   "message": "Requested time 23:00 is outside the  
5             doctor's working hours (10:00 - 18:00)"  
6 }
```

Patient cannot have two active appointments with the same doctor on the same day:

The screenshot shows an HTTP client interface. At the top, there's a navigation bar with "My Collection > Get data", "Save", and "Share" buttons. Below this, the request method is set to "POST" and the URL is "http://localhost:8080/appointments". A "Send" button is visible. The "Body" tab is selected, showing a JSON request body:

```
1 {
2   "patientId": 21,
3   "doctorId": 22,
4   "date": "2026-07-10",
5   "startTime": "11:00"
6 }
```

Below the request body, the response status is "409 Conflict" with a duration of "23 ms" and a size of "308 B". The response body is shown in JSON format:

```
1 {
2   "status": 409,
3   "error": "Conflict",
4   "message": "Patient already has an appointment
5             with another doctor between 11:00 and 11:30
6             on 2026-07-10"
7 }
```


Patient cannot be booked with any doctor in an overlapping time range with another appointment:

The screenshot shows a REST client interface with the following details:

- Request:**
 - Method: **POST**
 - URL: `http://localhost:8080/appointments`
 - Body (JSON):

```
1 {
2   "patientId": 1,
3   "doctorId": 22,
4   "date": "2026-07-15",
5   "startTime": "11:00"
6 }
```
- Response:**
 - Status: **409 Conflict** (18 ms, 278 B)
 - Body (JSON):

```
1 {
2   "status": 409,
3   "error": "Conflict",
4   "message": "Doctor is already booked between
               11:00 and 11:30 on 2026-07-15"
5 }
```

Cannot record two visits for the same appointment :

The screenshot shows a REST client interface with a tab labeled "[CONFLICT] POST Get da". The URL bar shows "http://localhost:8080/appointments/21...". The request body is a JSON object:

```
1 {
2   "diagnosis": "Hypertension Stage 1",
3   "prescription": "Amlodipine 5mg once daily for
4     30 days"
5 }
```

The response status is "409 Conflict" with a duration of 14 ms and a size of 268 B. The response body is a JSON object:

```
1 {
2   "status": 409,
3   "error": "Conflict",
4   "message": "A visit record already exists for
5     appointment ID: 21"
6 }
```

patients are not able to book or reschedule in between a pre-existing appointment time slot:

The screenshot displays a REST client interface with the following components:

- Header:** Shows a status of **[CONFLICT]** for a **POST** request to **Get da**. The environment is set to **No environment**.
- Toolbar:** Includes **Save** and **Share** buttons.
- Request Section:**
 - Method:** **POST**
 - URL:** **http://localhost:8080/appointments**
 - Body:** A JSON object:

```
{  "patientId": 2,  "doctorId": 22,  "date": "2026-07-15",  "startTime": "11:10"}
```
 - Buttons:** **Send** and **Schema** (disabled).
- Response Section:**
 - Status:** **409 Conflict** (17 ms, 278 B)
 - Body:** A JSON object:

```
{  "status": 409,  "error": "Conflict",  "message": "Doctor is already booked between 11:10 and 11:40 on 2026-07-15"}
```